

Defining Object Inheritance (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-defining-object-inheritance>  <https://github.com/learn-co-curriculum/python-p3-defining-object-inheritance/issues/new>

Learning Goals

- Reduce repeated code and enhance objects using inheritance.
 - Define object inheritance in Python.
 - Write classes that inherit from **superclasses**.
- Accomplish complex programming tasks using knowledge from previous modules.

Key Vocab

- **Inheritance**: a tool that allows us to recycle code by creating a class that "inherits" the attributes and methods of a parent class.
- **Composition**: a tool that enables you to recycle code by adding objects to other objects. Rather than building on a base class as in inheritance, composition leverages the attributes and methods of an instance of another class.
- **Subclass**: a class that inherits from another class. Colloquially called a "child" class.
- **Superclass**: a class that is inherited by another class. Colloquially called a "parent" class.
- **Child**: another name for a subclass.
- **Parent**: another name for a superclass.
- **super()**: a built-in Python function that allows us to manipulate the attributes and methods of a superclass from the body of its subclass.
- **Decorator**: syntax that allows us to add functionality to an object without modifying its structure.

Introduction

In the real-world, different entities (people, animals, cars, you name it) are related in various ways. Within a single entity or group, there exist systems of classification. For example, the "dogs" entity or category includes pugs, corgis, labs, etc. All of these breeds share common features because they are all dogs. But they all have certain unique traits as well.

Another example: you are writing a web application in which users are either admins, instructors or students. All of these entities are "users" and have common features, but they all have some unique traits as well.

How can our code reflect that fact that these different categories of things all share some, or even many, characteristics but all have some unique attributes as well? Well, we could write separate `Admin`, `Instructor` and `Student` classes that each contain repetitious code to lend each of these classes shared attributes and behaviors. We know, however, that repetitive code is always something to be avoided. Not only is it time consuming, but what happens when we need to make a change to this shared behavior? We'd have to code the same change in three places.

Instead, we can use **inheritance**. The use of inheritance allows us to create a family of classes with shared behavior, while still differentiating those classes. With inheritance, we could *inherit* the `Admin`, `Instructor` and `Student` classes from a `User` class. Then, any changes made to the `User` class would apply to the other class.

While you may not write your own classes that use inheritance very frequently, you will encounter it frequently as a developer, particularly when working with other libraries (such as SQLAlchemy, which you'll learn later in this phase). Once we introduce the use of databases and the challenge of connecting our programs to our database, you'll encounter inheritance in nearly every program you write for the web. More on that later.

What is Inheritance?

In Python, classes can inherit from one another. This means that they acquire all of the attributes and behaviors (i.e. all of the methods) of the parent, also called the **superclass**. An object can directly access public inherited attributes and methods. However, since private attributes are only visible in the defining class, the private inherited attributes must be accessed through public inherited methods.

In this exercise, we'll be building our own chain of inheritance.

Code Along: Basic Inheritance

In this domain model, we have class `Vehicle` that will act as the **parent**, or **superclass**. We will create **child** classes, also known as **subclasses** for different types of `Vehicle`s, such as car.

Step 1: Defining the Superclass

Open up `lib/vehicle.py`. We're going to define some methods in this parent class so that our subclasses, when we make them, will have access to them.

```
class Vehicle:

    def __init__(self, wheel_size, wheel_number):
        self.wheel_size = wheel_size
        self.wheel_number = wheel_number

    def go(self):
        return "vrrrrrrrrrooom!"

    def fill_up_tank(self):
        return "filling up!"
```

Instances of `Vehicle` initialize with a wheel size and number. We also have `go()` and `fill_up_tank()` instance methods that describe some common vehicle behavior.

Go ahead and paste the above `Vehicle` class code into your `Vehicle` class, and run the test suite and you'll see that you are passing all of the tests for the `Vehicle` class but none of the tests for the `Car` class.

Step 2: Defining the Subclass

Open up `lib/car.py` . Notice that we are requiring `lib/vehicle.py` . That is because our `Car` class will need access to the `Vehicle` class and will therefore need access to the file that contains that class.

Go ahead and define the class in the following way:

```
class Car(Vehicle):  
    pass
```

We use `Vehicle` as an argument for the `Car` class to note that `Car` inherits from `Vehicle` . Run the test suite again and you'll see that you are passing a number of tests for the `Car` class.

Wow! We didn't write *anything* in our `Car` class but instances of `Car` class *inherit* all of the `Vehicle` attributes and methods and therefore have access to them.

► *Which arguments are required to instantiate a new `Car` object?*

We're still failing the `go()` test however. Looks like the test is expecting the `go()` method on an individual car to return the phrase:

"VRRR000000000000000000000000M!!!!!" . This is different than the return value of the `go()` method that we inherited from the `Vehicle` class.

Let's overwrite the inherited `go()` method with one specific to `Car` .

Step 3: Overwriting Inherited Methods

In `lib/car.py` , write the following method:

```
class Car(Vehicle):  
    def go(self):  
        return "VRRR000000000000000000000000M!!!!!"
```

Now, run the tests again and you should be passing all of them.

Class Introspection

How does our above example work? Well, when your program is being executed, at the point at which the `go()` method is invoked, the interpreter will first look in the class to which the instance of car that we are calling the method on belongs. If it finds a `go()` method there, it will execute *that method*. If it doesn't find such a method there, it will move on to look in the parent class that this class inherits from.

You can see how Python classes inherit from one another by using Python to do some *introspection* on our classes.

Navigate to the `lib` folder. Open up the Python shell, and start by importing the classes we built there:

```
from vehicle import Vehicle
from car import Car
```

This will let you interact with the code you've written in those files from within the Python shell.

NOTE: We'll dive deeper into `import` in the "*Configuring Python Applications*" module.

We can ask the `Car` class what its parent, or "superclass" is (what class the `Car` class inherits from) with its `__bases__` attribute:

```
Car.__bases__
# (<class 'vehicle.Vehicle'>,)
```

NOTE: the `__bases__` attribute is a `tuple`. This is because it shows *all* superclasses of the `Car` class. While there is only one here, there are many classes that inherit directly from multiple parent classes.

How does this work? How can we access the `__bases__` attribute of our `Car` class, even though we didn't define it ourselves? The `__bases__` attribute is available on all Python classes, even built-in ones like the `int` class:

```
int.__bases__
# (<class 'object'>,,)
```

That's because all Python classes share the same metaclass: the `type` class [↗\(https://realpython.com/python-metaclasses/#:%7E:text=type%20is%20a%20metaclass%2C%20of,instance%20of%20the%20type%20metaclass.\)!](https://realpython.com/python-metaclasses/#:%7E:text=type%20is%20a%20metaclass%2C%20of,instance%20of%20the%20type%20metaclass.)!)

```
Car.__class__  
# <class 'type'>  
int.__class__  
# <class 'type'>
```

Conclusion

We've seen how to set up inheritance to share behavior from one class to another using parent classes as arguments in our class definition (`class Child(Parent)`), which lets the subclass acquire attributes and methods that are defined on the parent class. We also discussed how **class introspection** works in Python, when multiple classes define the same method.

Resources

- [Python 3.8 Documentation](https://docs.python.org/3.8/) [↗\(https://docs.python.org/3.8/\)](https://docs.python.org/3.8/)
- [Inheritance - Python](https://docs.python.org/3/tutorial/classes.html#inheritance) [↗\(https://docs.python.org/3/tutorial/classes.html#inheritance\)](https://docs.python.org/3/tutorial/classes.html#inheritance)
- [Inheritance and Composition: A Python OOP Guide - Real Python](https://realpython.com/inheritance-composition-python/) [↗\(https://realpython.com/inheritance-composition-python/\)](https://realpython.com/inheritance-composition-python/)
- [Python Metaclasses - Real Python](https://realpython.com/python-metaclasses) [↗\(https://realpython.com/python-metaclasses\)](https://realpython.com/python-metaclasses)

This tool needs to be loaded in a new browser window

Load Defining Object Inheritance (CodeGrade) in a new window